

LIME & Robustness / Certification

From Data to Solutions
ETH Zürich · 16. July 2026

Christina Tsakanika



Agenda

1

Motivation

Clever Hans · don't trust performance blindly

2

Interpretability

what it is, why it matters, when it doesn't

3

A taxonomy of methods

by-design vs post-hoc · where LIME sits

4

From CP & ICE to LIME

the limitation that forces the jump

5

LIME in depth

intuition, algorithm, the objective, data types

6

Limitations

kernel width, instability → bridge to SHAP

7

Local → global: SP-LIME

2 trust problems, the model-choice study

8

Robustness & certification

fooling LIME, provable stability

01

Motivation

- A high score is not a reason.
- Models can be right for the wrong reasons.

The original “Clever Hans”

A cautionary tale from 1900s Berlin

“

The horse that could “do arithmetic” ... until someone controlled the setup and found it was reading its trainer’s body language.

The claim: Hans tapped out answers to sums, dates, musical intervals.

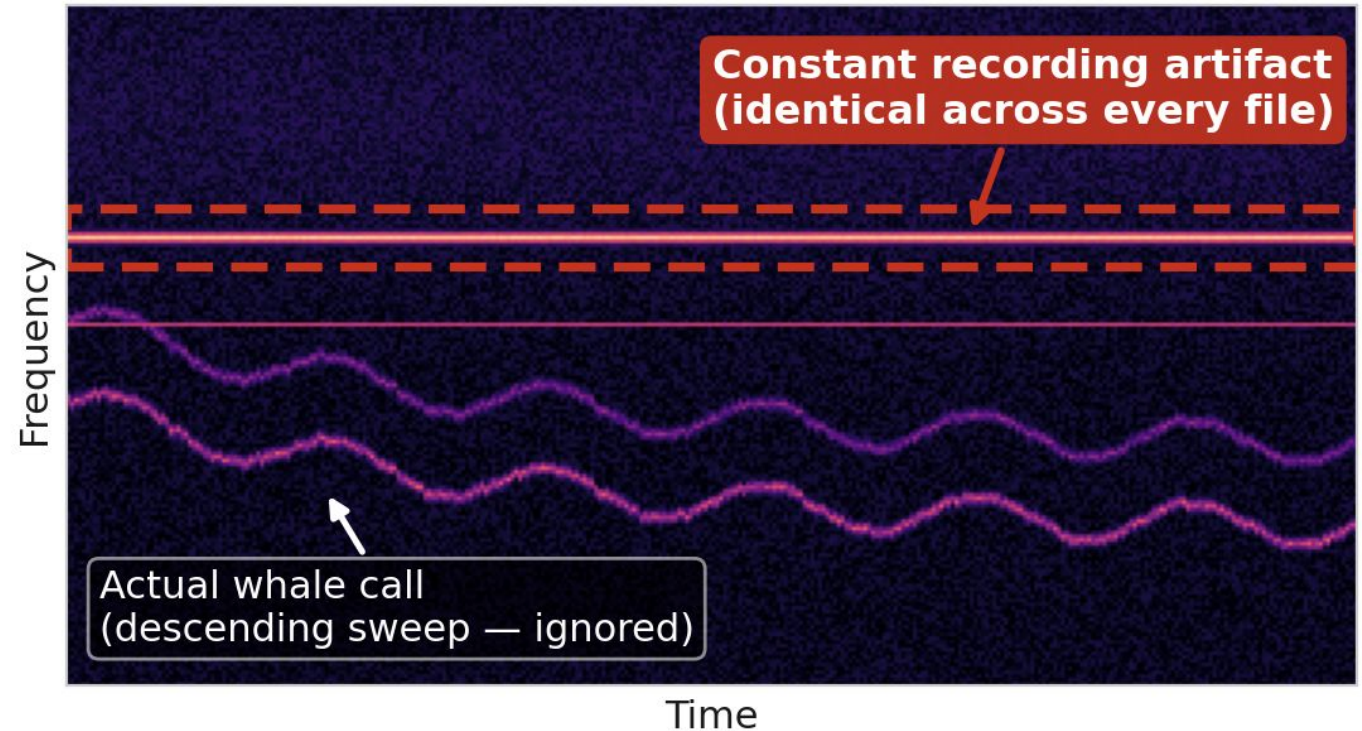
The reality: he had learned to stop tapping on subtle, unconscious cues from whoever knew the answer.

The lesson: correct outputs, wrong mechanism and no one noticed until the experiment was designed to catch it.

The whale that wasn't there

A whale-call detector scored high on the leaderboard, then failed on new recordings.

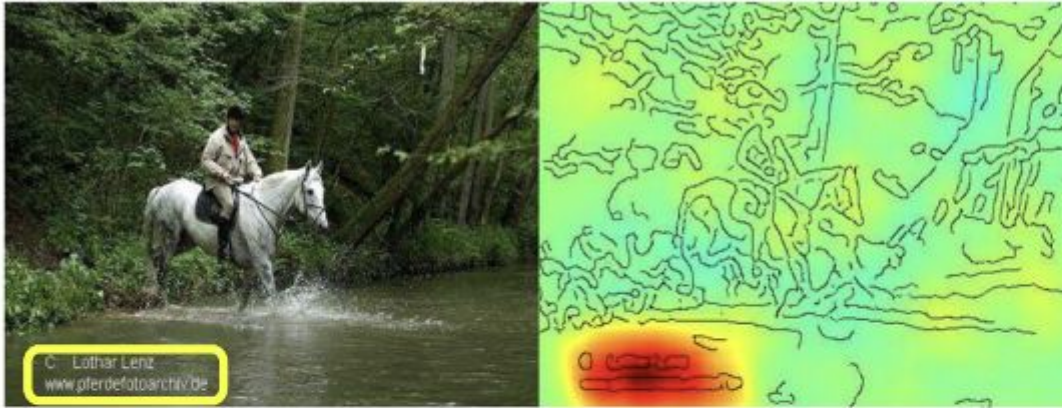
It had locked onto a constant artifact present in every training clip, ignoring the actual call.



Source: DeLMA & Cukierski, 2013 (Kaggle Whale Detection)

Reading the caption, not the image

Horse-picture from Pascal VOC data set



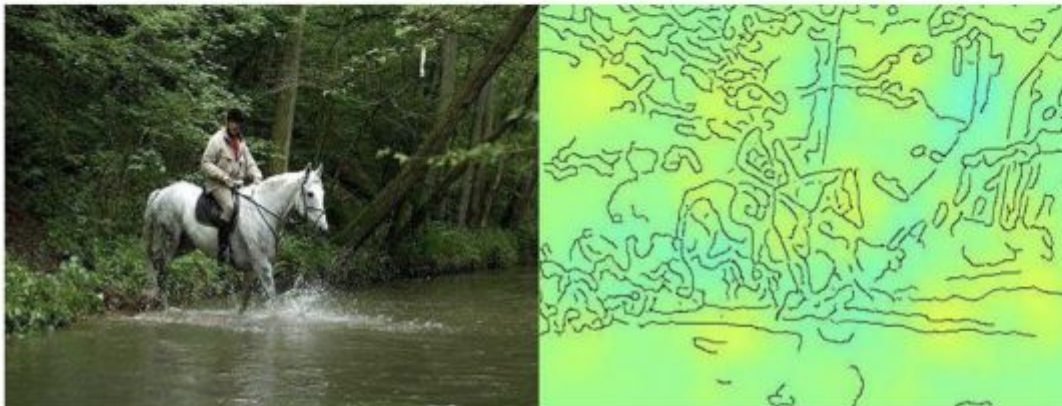
Source tag
present



Classified
as horse

A classifier appeared to recognise the object, but its evidence sat on a source watermark printed in the corner.

Remove the tag and the prediction falls apart.



No source
tag present



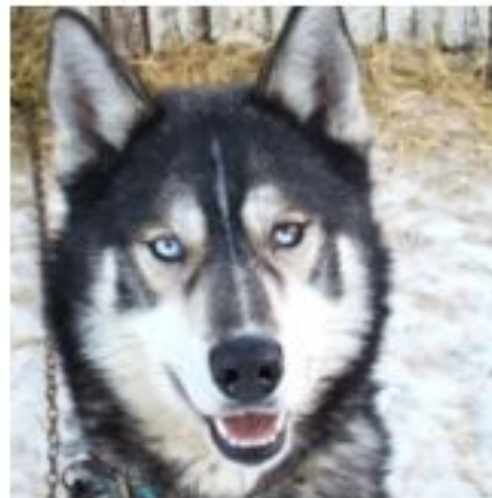
Not classified
as horse

Source: Lapuschkin et al., 2019 (*Nature Communications*)

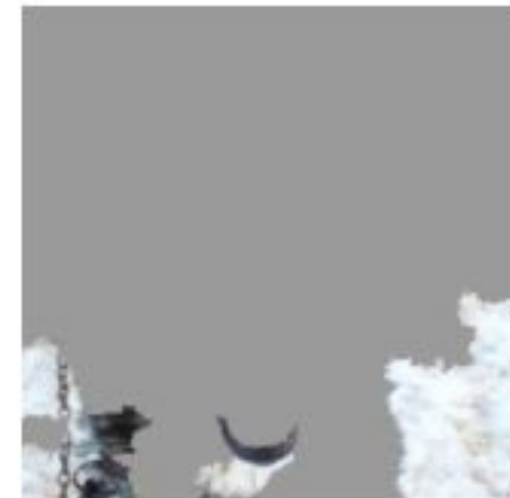
■ A snow detector in disguise

The now-classic LIME example: a husky is called a “wolf.”

The explanation reveals the model looked at the snowy background, not the animal.



(a) Husky classified as wolf



(b) Explanation

Source: Ribeiro, Singh & Guestrin, 2016
(the LIME paper)

Why held-out accuracy isn’t enough

- **Same distribution**

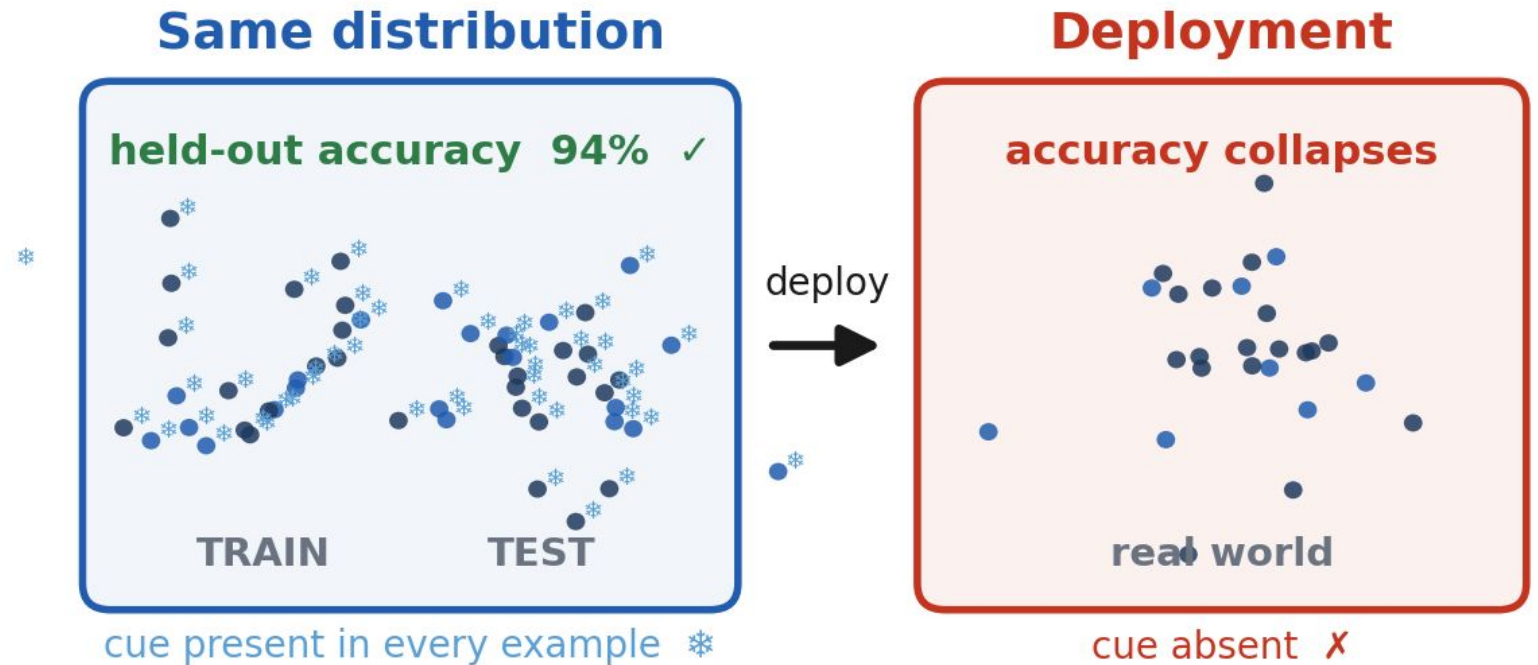
A held-out split only tests the distribution the model was trained on.

- **Shortcuts survive**

Spurious cues (snow, captions, artifacts) are present at test time too.

The blind spot only opens after deployment.

In deployment the cue disappeared and so did the accuracy.



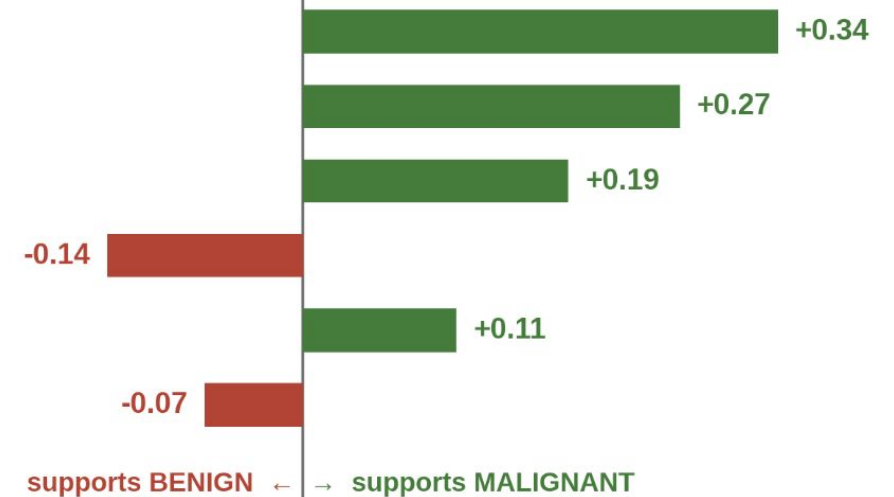
From WHAT to WHY



worst perimeter = 155.1
worst concave points = 0.187
worst area = 1575
mean smoothness = 0.089
mean texture = 24.5
mean symmetry = 0.181

WHY which inputs drove it, and by how much

each bar = the effect of KEEPING that feature at the patient's value



Catch spurious correlations
before deployment

Audit for bias

Build justified trust

02

What is interpretability?

- Definitions, and the fuzzy line to “explainability”.
- Why it matters — and when it doesn’t.

What is interpretability?

Two working definitions

“

The degree to which a human can understand the cause of a decision.

Miller (2017)

“

A method is interpretable if a user can correctly and efficiently predict the method's results.

Kim, Khanna & Koyejo (2016)

Interpretability vs. Explainability

I Interpretability

- A property of the model itself.
- Intrinsic transparency
- e.g. a short decision tree, a linear model.

E Explainability (XAI)

- A post-hoc artifact about a model's output.
- An explanation justifies a decision after the fact.
- e.g. LIME / SHAP on top of a black box.

In practice the terms are used almost interchangeably.

When interpretability and when not

When we need interpretability

- **Debugging** → catch spurious shortcuts
- **Trust** → justify acting on a high-stakes prediction
- **Fairness & audit** → check the model isn't using protected attributes
- **Science** → turn a predictive model into insight
- **Regulation** → GDPR-style “right to an explanation”, credit, health

When we don't

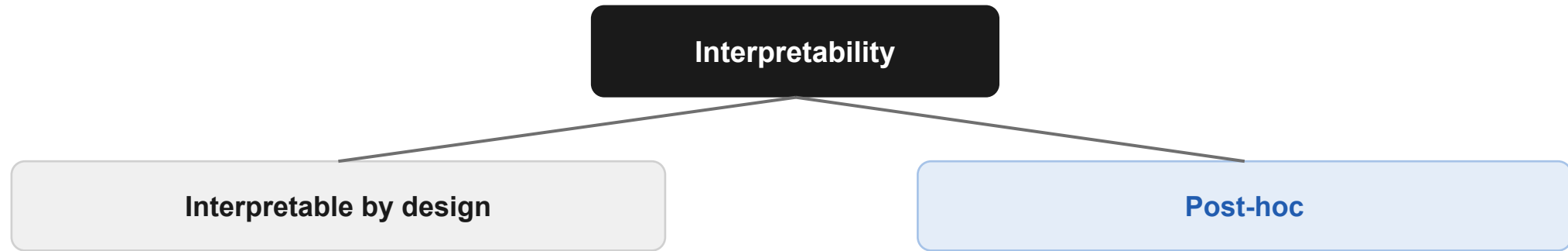
- Low-stakes decisions with no cost of error.
- Very well-studied, trusted problems.

03

A taxonomy of methods

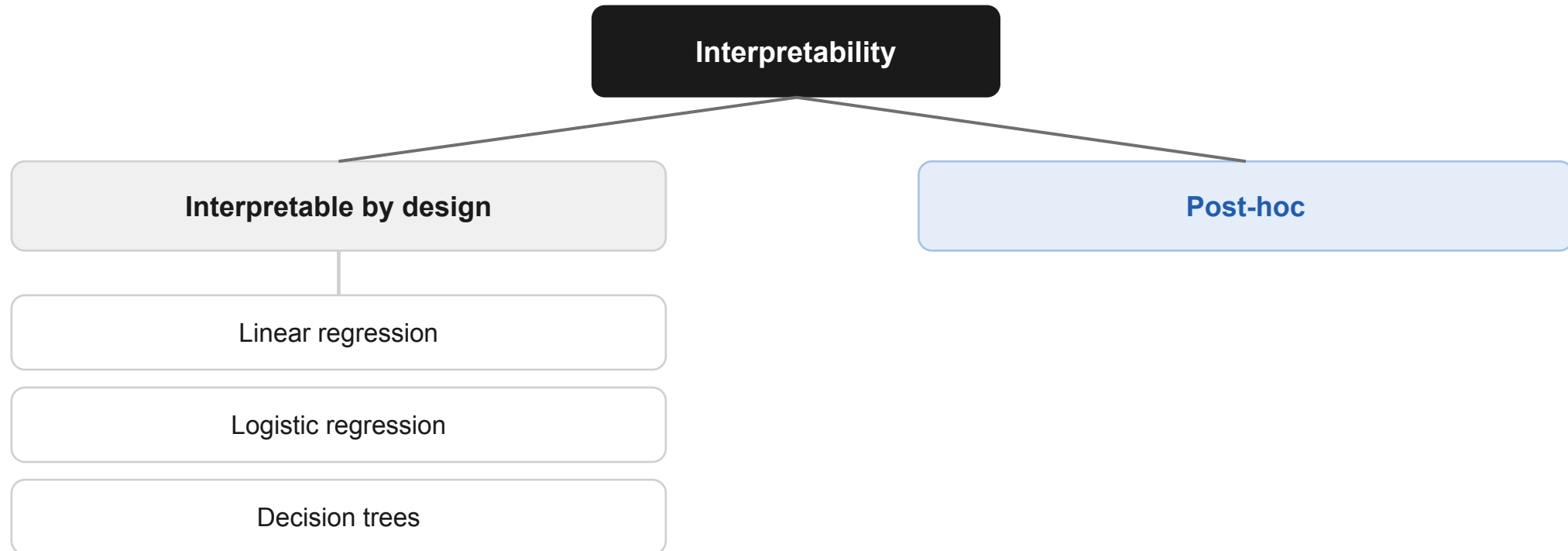
A taxonomy of interpretability methods

LIME = post-hoc · model-agnostic · local



A taxonomy of interpretability methods

LIME = post-hoc · model-agnostic · local



A quick recap

Linear regression

$$\hat{y} = \beta_0 + \sum \beta_i x_i$$

Prediction is a weighted sum.
Each β_i is the feature's marginal effect
read the weight, know the contribution.

Logistic regression

$$\text{log-odds} = \beta_0 + \sum \beta_i x_i$$

Same additive structure, now on the
log-odds. Each weight moves the odds
up or down by a fixed factor.

Decision trees

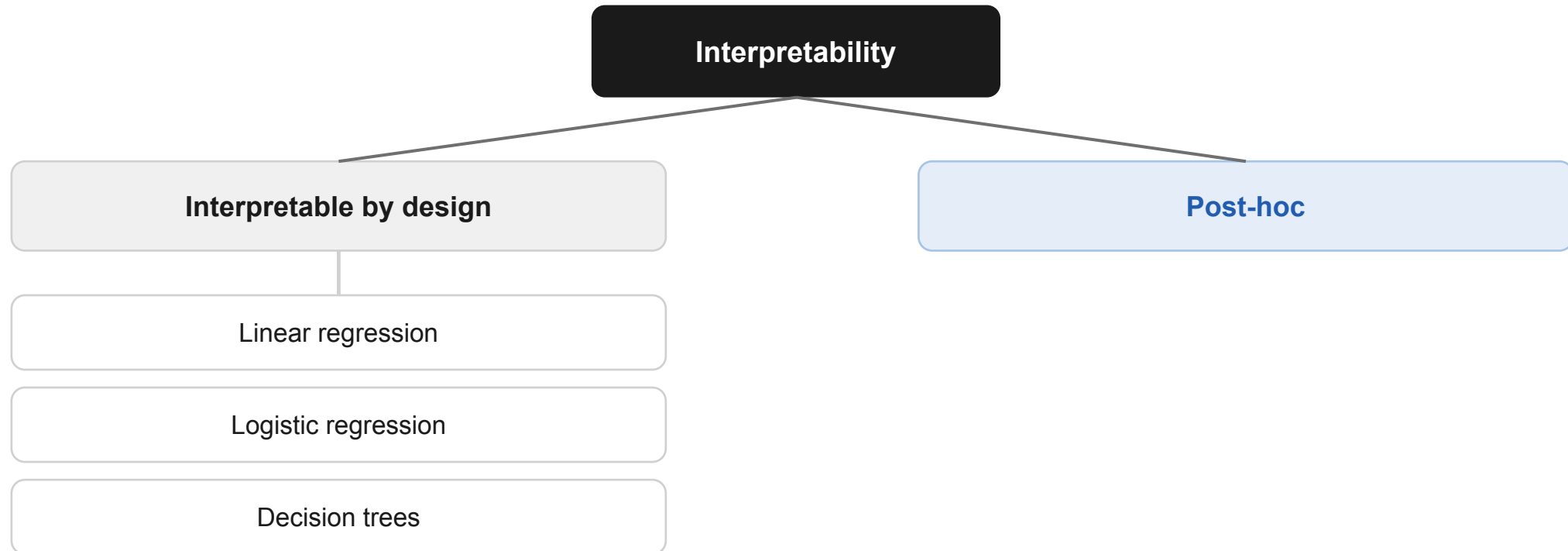
`if ... then ... else`

The prediction is a readable path of
if-then splits from root to leaf.

Cost of transparency: these models trade some accuracy on complex data for a mechanism a human can read. LIME lets us keep the accurate black box and recover a local, readable explanation.

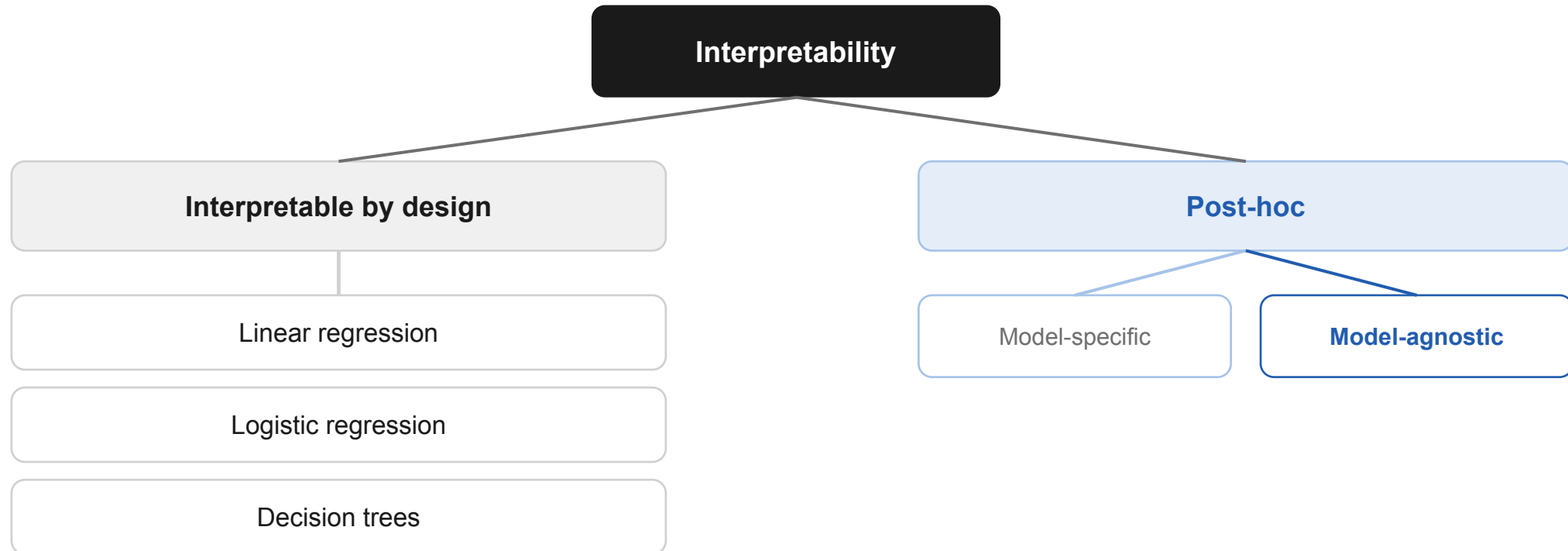
A taxonomy of interpretability methods

LIME = post-hoc · model-agnostic · local



A taxonomy of interpretability methods

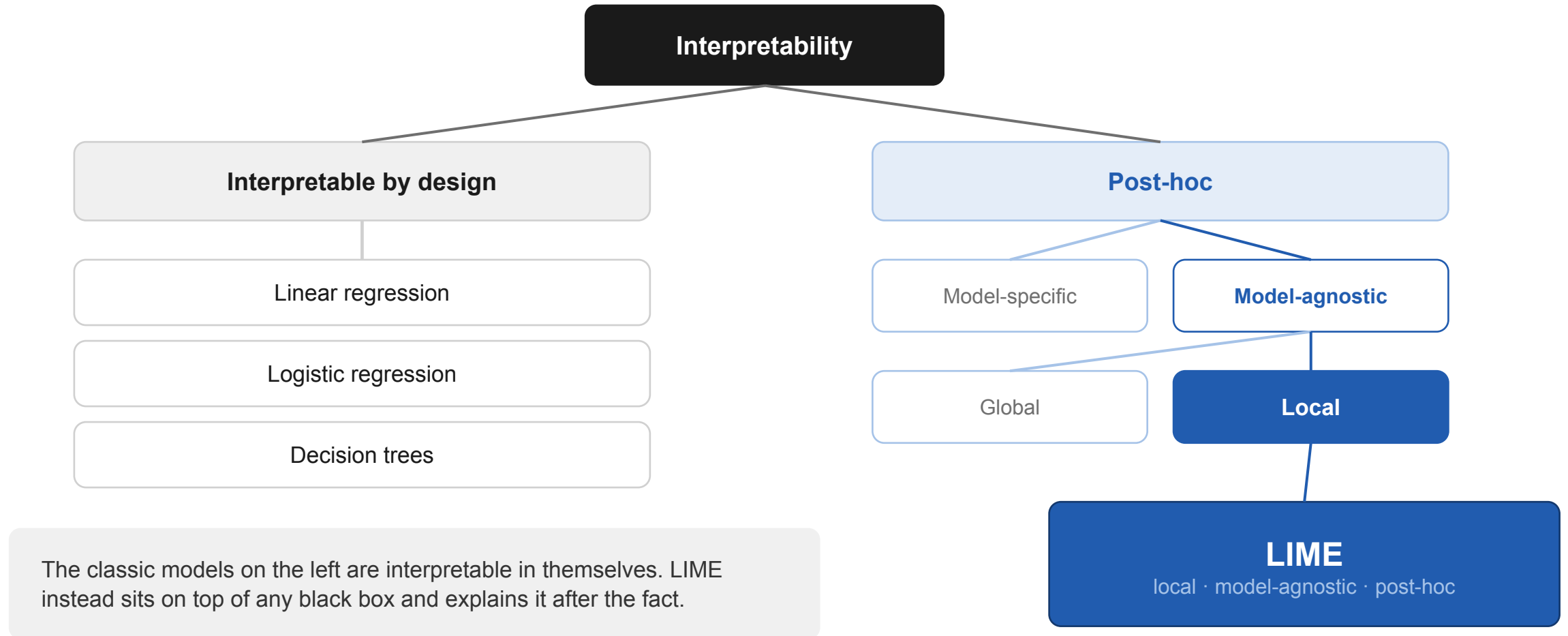
LIME = post-hoc · model-agnostic · local



The classic models on the left are interpretable in themselves. LIME instead sits on top of any black box and explains it after the fact.

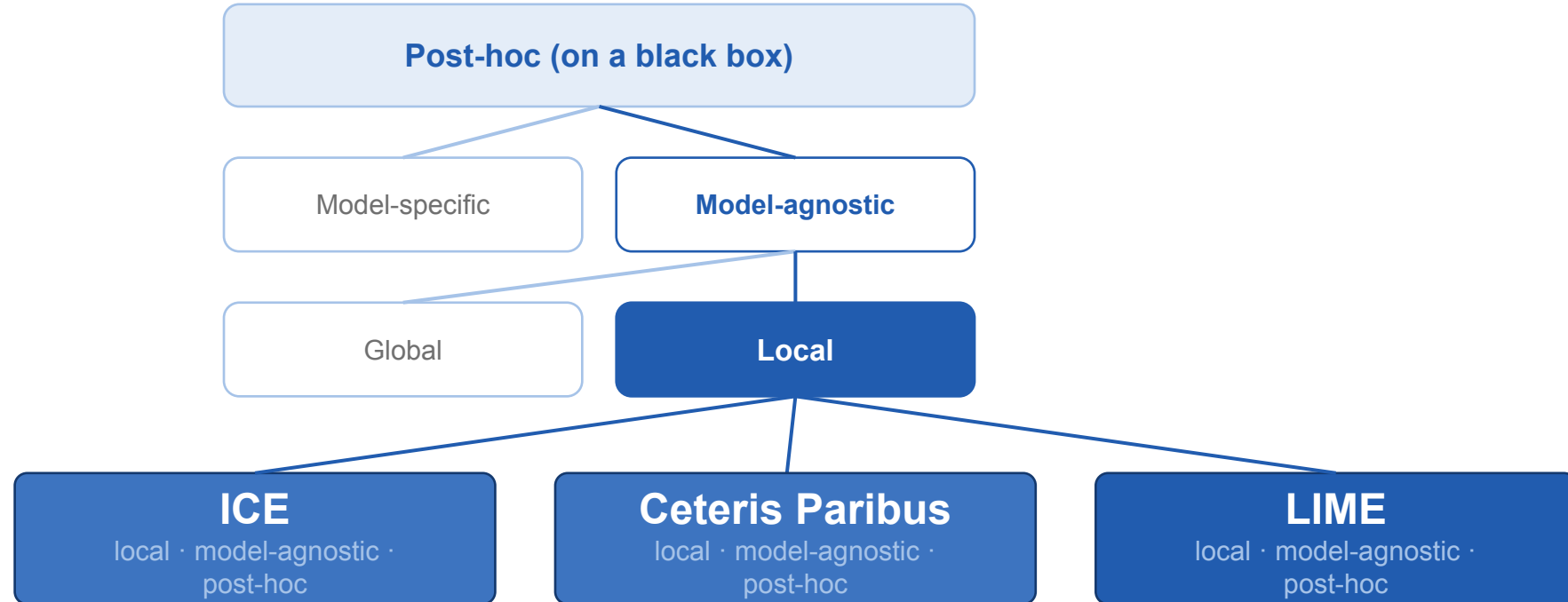
A taxonomy of interpretability methods

LIME = post-hoc · model-agnostic · local



A taxonomy of interpretability methods

LIME = post-hoc · model-agnostic · local



04

From ceteris-paribus & ICE to LIME

- Both vary one feature at a time.
- That limitation is exactly what LIME removes.

Ceteris-paribus & ICE plots

One feature at a time

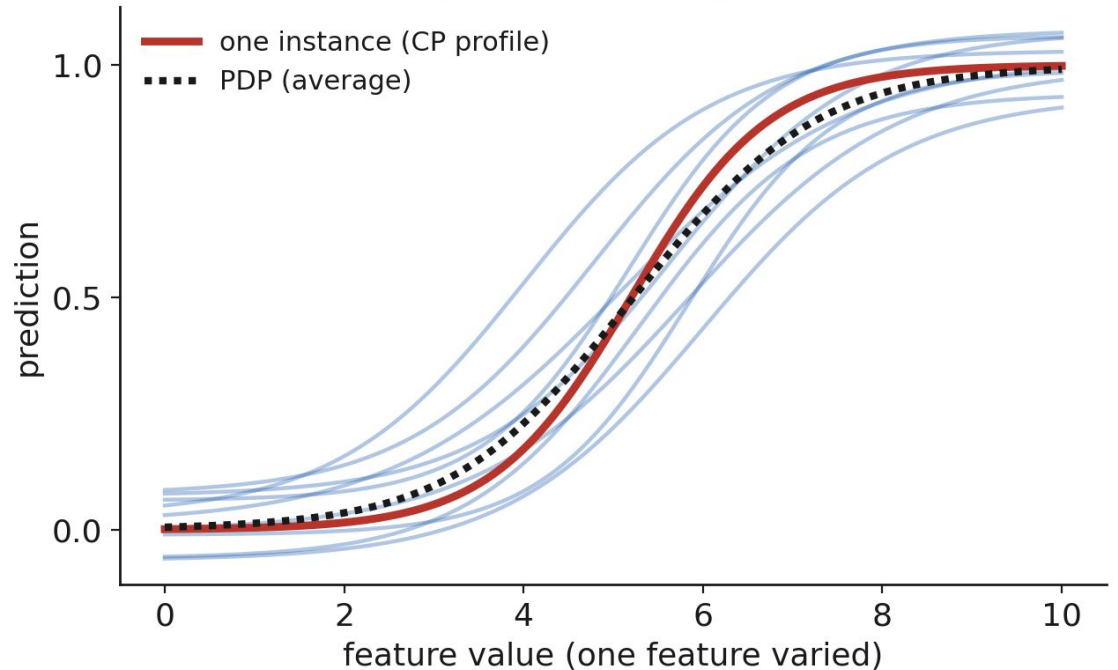
Ceteris paribus: vary one feature, hold the rest fixed, watch the prediction move.

ICE: one such curve per instance, stacked reveals heterogeneity the PDP average hides.

Great for: seeing how a single feature drives the model.

But: it isolates *one axis*. To reason about a single prediction you'd need one plot per feature ...

ICE: one ceteris-paribus curve per instance

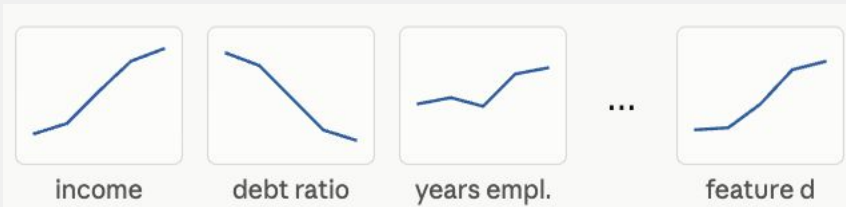


From CP / ICE to LIME

2 Limitations

1.d plots, one prediction. With many features we get no single, compact answer to “why this output?”

E.g Loan Applicant (d-features)

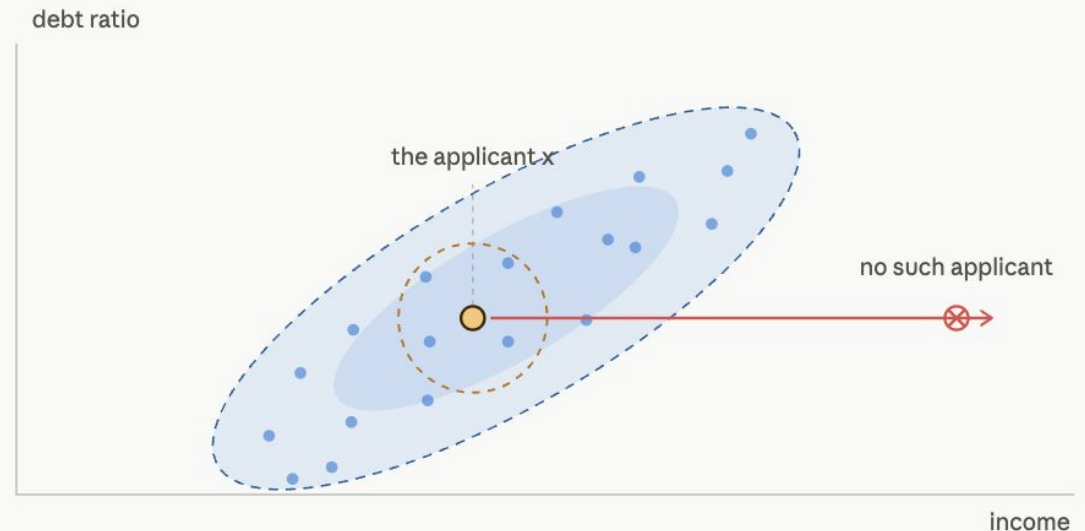


d features, d plots for 1 decision

2.Off-manifold. Holding correlated features fixed while sweeping one creates unrealistic, impossible instances the curve leaves the data.

The LIME idea

What if, instead of one axis at a time, we perturb everything at once in a small neighborhood and fit one simple model there?

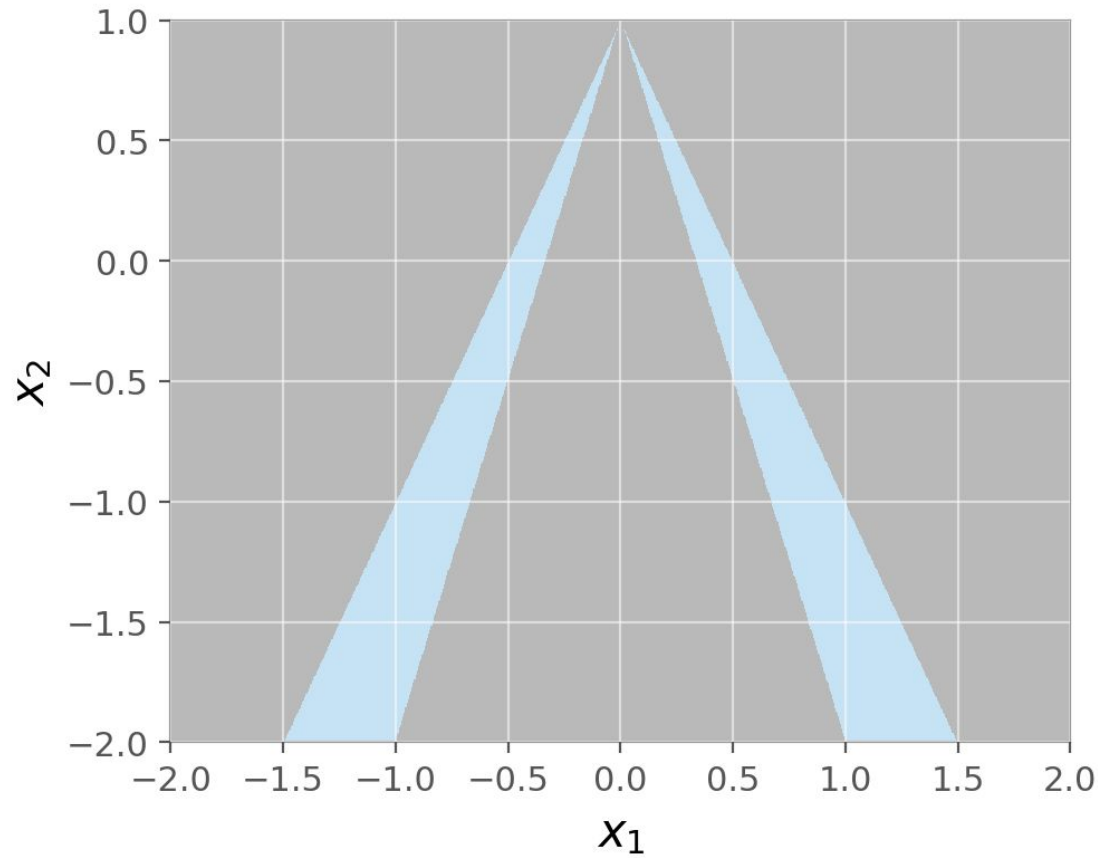


05

LIME in depth

- Intuition · algorithm · the objective · data types.
- Ribeiro, Singh & Guestrin (2016).

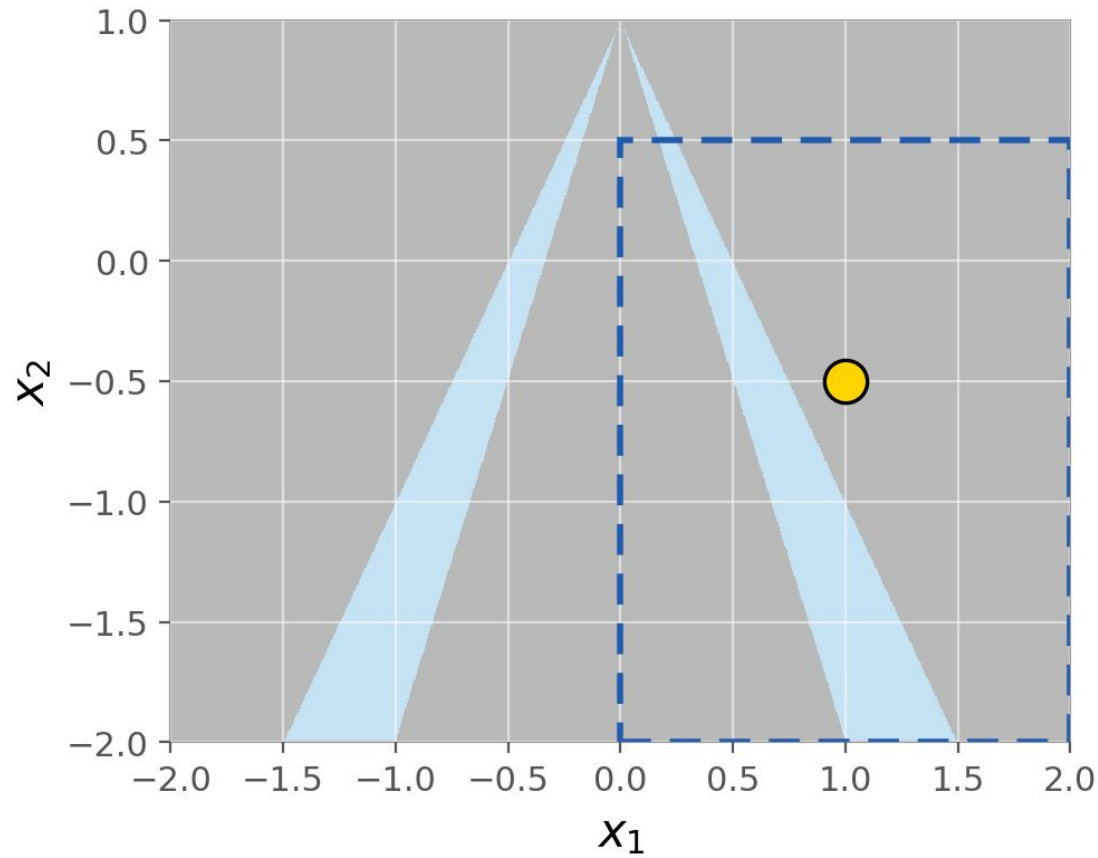
Local · Interpretable · Model-agnostic Explanations



This is f , the function we want to explain.

Grey is one class, blue is the other. The boundary is curved and it has two branches.

Local · Interpretable · Model-agnostic Explanations



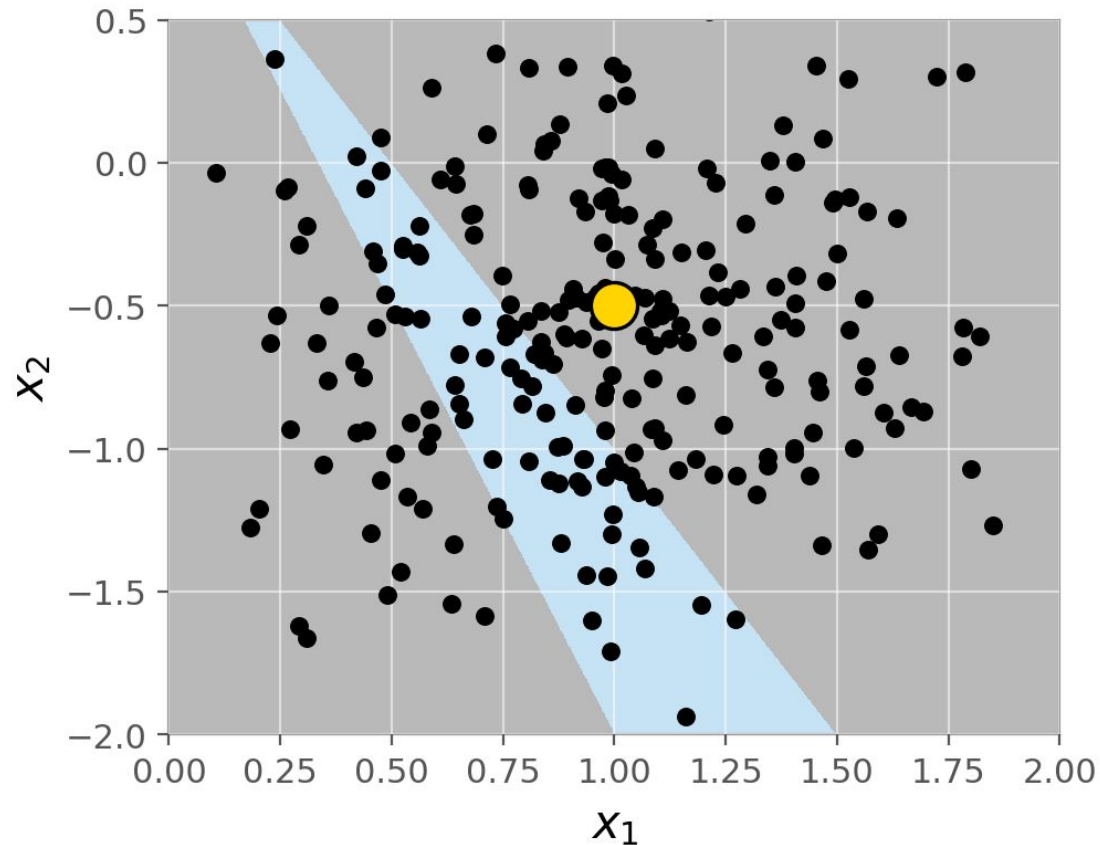
We do not explain **f**.

We explain **one prediction** the yellow point.

Everything from here happens inside the dashed box.

Local · Interpretable · Model-agnostic Explanations

Sample around it and ask f for prediction

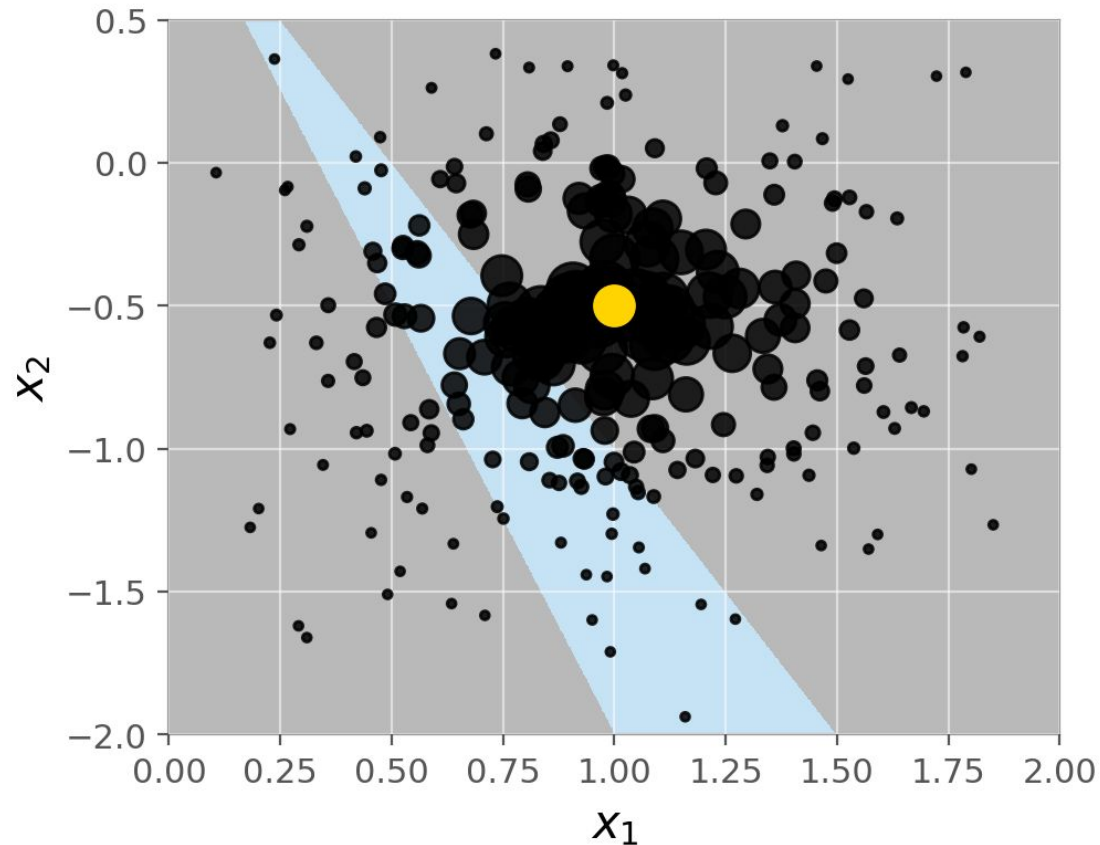


Generate many points near the instance.

Ask f for a label for each one.

Local · Interpretable · Model-agnostic Explanations

Weight by proximity



Now weight each sample by how close it is.

Big dot = counts a lot.

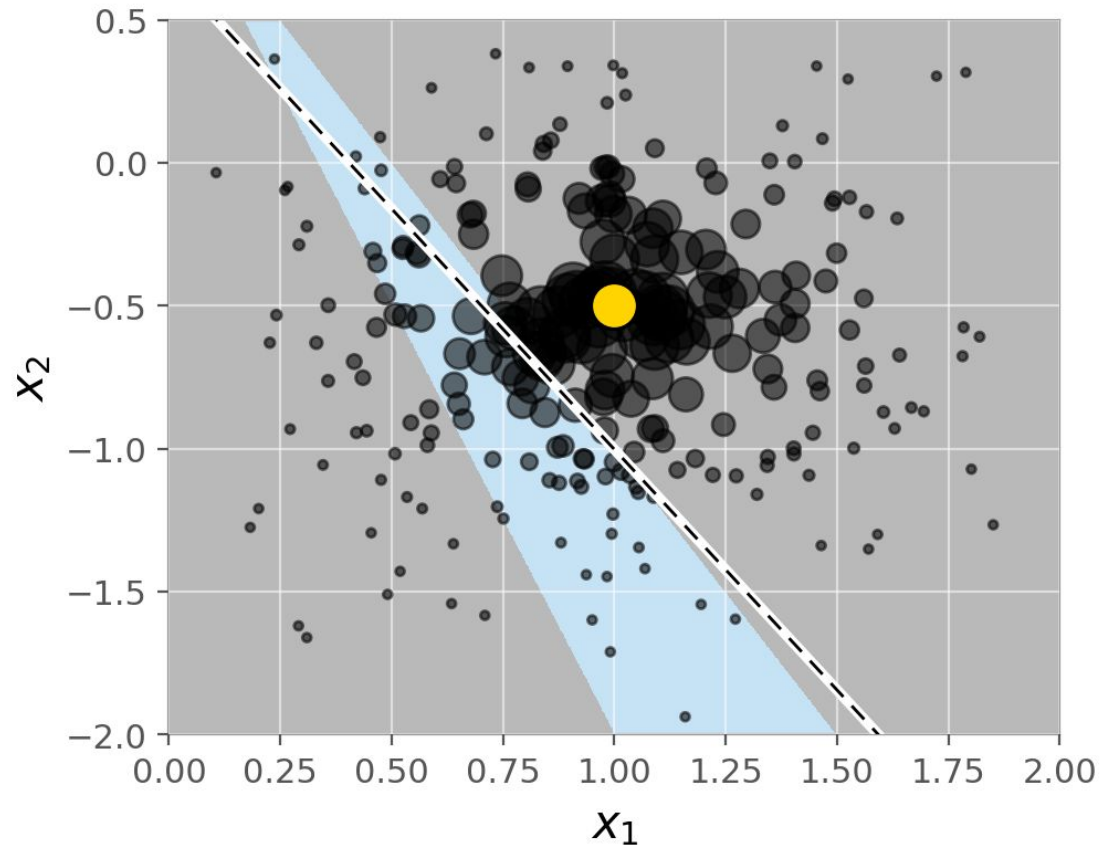
Small dot = almost ignored.

The kernel π decides what "near" means.

$$\pi_x(z) = \exp\left(-\frac{D(x,z)^2}{\sigma^2}\right)$$

Local · Interpretable · Model-agnostic Explanations

Fit one straight line

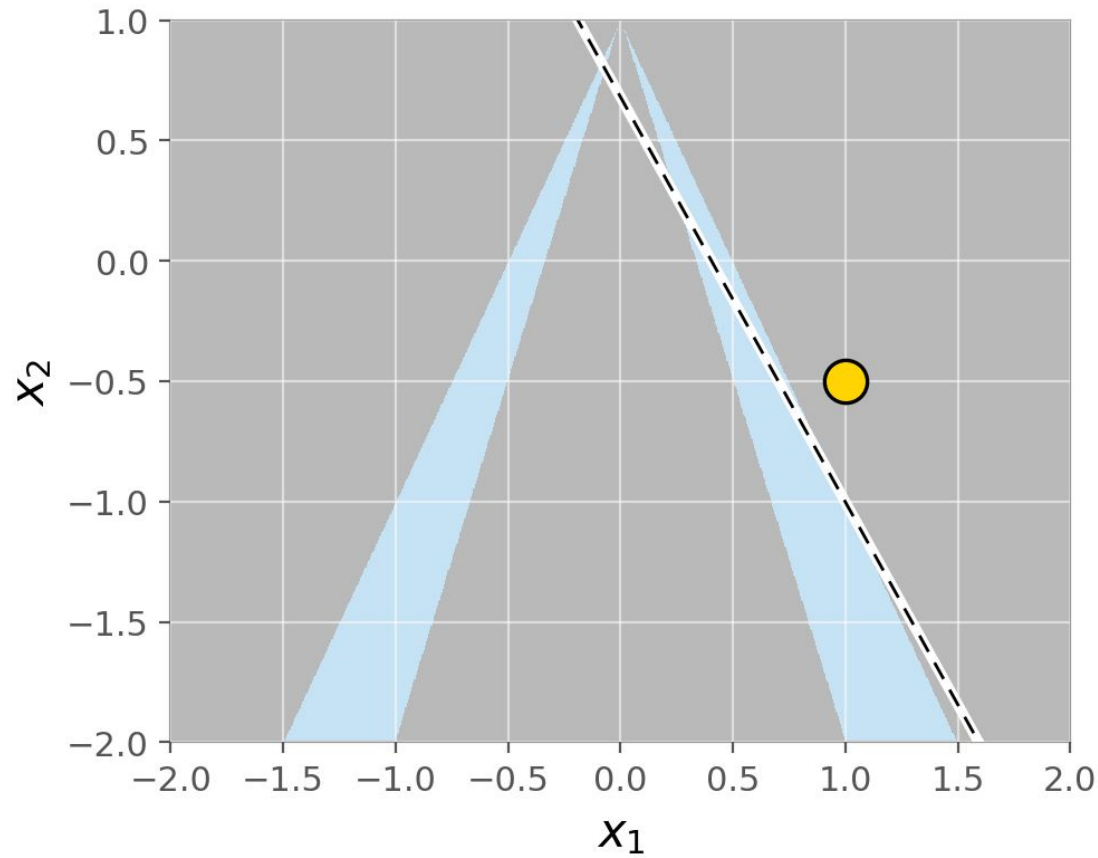


Fit a single straight line through the **weighted** samples. That line is **g**, the explanation.

Near the yellow point, the boundary looks like a straight edge. A line can copy that.

Local · Interpretable · Model-agnostic Explanations

Locally faithful. Globally wrong.



Same line. Zoomed back out.

It ignores the entire left branch. It claims half the space is grey.

As a global description of f , it is not accurate.

Careful: here x has 2 features, we can draw everything. A photo has no axes to draw. That is the next section.

What we are trying to do?

"By explaining a prediction, we mean presenting textual or visual artifacts that provide qualitative understanding of the relationship between the instance's components and the model's prediction."

Ribeiro, Singh & Guestrin (2016), §2

Individual

We explain one prediction of a black box model.

Not the model. One row, one photo, one sentence.

Interpretable

The explanation must be readable by a person

Faithful

It must match what the model actually does near this instance.

"interpretable" is a claim about the **reader**, not about the model.
what exactly is a person supposed to read?

Our instances

x: the raw representation, chosen for the model

Instance	Example	Raw representation — input in model f	Dimension
Text	"interpretability is the most interesting ml topic"		
Image	a photo of a wolf		
Tabular	loan applicant		

Our instances

x: the raw representation, chosen for the model

Instance	Example	Raw representation — input in model f	Dimension
Text	"interpretability is the most interesting ml topic"	one sentence embedding [0.0099, -1.284, ... , 8.0000]	$\mathbb{R}^{768}$
Image	a photo of a wolf	pixel tensor, 3 colour channels per pixel [[[0.21, 0.19, 0.44], ...]]	$\mathbb{R}^{27 \times 27 \times 3} = \mathbb{R}^{2187}$
Tabular	loan applicant	the feature values themselves [46.0, 0.66, 3.0]	$\mathbb{R}^3$

Features and Interpretable data representations

The model's features were chosen for accuracy. An explanation's features must be chosen for a person.

Raw representation

$$\mathbf{x} \in \mathbb{R}^d$$

Numbers, pixels, embeddings.

Whatever the model was trained on.

This is the domain of f .

Interpretable representation

$$\mathbf{x}' \in \{0, 1\}^{d'}$$

A row of “switches”

This is the domain of g .

for text classification: a binary vector indicating the presence or absence of a word

for image classification: a binary vector indicating the “presence” or “absence” of a super-pixel

$\mathbf{g} \in \mathbf{G}$. $\mathbf{G}$ is the family of interpretable models. We take the sparse linear ones: $\mathbf{g}(\mathbf{z}') = \mathbf{w} \cdot \mathbf{z}'$

The domain of g is $\{0,1\}^{d'}$ g acts over absence/presence of the interpretable components.

From x to x'

Instance	Raw x	d	One switch =	x'	d'
Text	sentence embedding [0.0099, -1.284, ... , 8.0000]	$\mathbb{R}^{768}$	one word in this sentence "interpretability is the most interesting ml topic"	[1,1,1,1,1,1,1]	$\{0,1\}'$
Image	pixel tensor [[[0.21, 0.19, 0.44], ...]]	$\mathbb{R}^{2187}$	one super-pixel	[1,1,1, ... ,1]	$\{0,1\}^{50}$
Tabular	the three values [46.0, 0.66, 3.0]	$\mathbb{R}^3$	this feature is at its value	[1,1,1]	$\{0,1\}^3$

Why must x' be all ones? Because we defined **0 = removed**. So the instance with nothing removed can only be a row of 1s.

LIME for text

Interpretable representation = word present / absent

“The plot was **predictable** and **dull**, but the **soundtrack** was genuinely **beautiful**.”

■ pushes toward NEGATIVE ■ pushes toward POSITIVE

Mechanism: toggle words on / off, re-query the model

	dull	predictable	beautiful	soundtrack
x' (original)	1	1	1	1
z'_1	0	1	1	1
z'_2	1	1	0	1
z'_3	0	1	0	1

The objective

Fidelity and interpretability, traded off

$$\xi(x) = \operatorname{argmin}_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g)$$

The loss

$$\mathcal{L}(f, g, \pi_x) = \sum_{z, z' \in \mathcal{Z}} \pi_x(z) (f(z) - g(z'))^2$$

$\mathcal{L}(f, g, \pi_x)$

Local infidelity

How far g drifts from f measured **only in the neighbourhood** that π_x defines.

$\Omega(g)$

Complexity

How hard g is to read. For a linear g : the number of non-zero weights. **A hard cap at K**

$g \in G$

Interpretable family

Sparse linear models. We take $g(z') = w \cdot z'$.

Algorithm 1 sparse linear explanations

Step 1: what you must bring

```
Require: Classifier  $f$ , Number of samples  $N$   
Require: Instance  $x$ , and its interpretable version  $x'$   
Require: Similarity kernel  $\pi$ , Length of explanation  $K$ 
```

```
 $Z \leftarrow \{ \}$   
for  $i \in \{1, 2, 3, \dots, N\}$  do  
     $z'_i \leftarrow \text{sample\_around}(x')$   
     $Z \leftarrow Z \cup \{ z'_i, f(z_i), \pi(z_i) \}$   
end for  
  
 $w \leftarrow \text{K-Lasso}(Z, K)$   
    ► features:  $z'_i$     target:  $f(z)$   
return  $w$ 
```

Require: instance x , and its interpretable version x'

Algorithm 1 sparse linear explanations

Step 2 the loop builds a dataset

Applicant x: 46 kCHF · 0.66 · 3 yrs → $f(x) = 0.004$ → DENIED

	z' (on / off)			z (actual input to f)					
	inc	debt	yrs	inc	debt	yrs	$f(z)$	D	π
x'	1	1	1	46.0	0.66	3.0	0.004	0.00	1.00
z'_1	0	1	1	85.1	0.66	3.0	0.548	1.00	0.55
z'_2	1	0	1	46.0	0.14	3.0	0.950	1.00	0.55
z'_3	1	1	0	46.0	0.66	8.8	0.038	1.00	0.55
z'_4	0	0	1	53.6	0.78	3.0	0.004	1.41	0.31

Require: Classifier f , Number of samples N
Require: Instance x , and its interpretable version x'
Require: Similarity kernel π , Length of explanation K

```
z ← { }  
for i ∈ {1, 2, 3, ..., N} do  
    z'ᵢ ← sample_around( x' )  
    z ← z ∪ { z'ᵢ , f(zᵢ) , π(zᵢ) }  
end for  
  
w ← K-Lasso( Z , K )  
    ► features: z'ᵢ    target: f(z)  
return w
```

1 = keep the applicant's value · 0 = resample it.

$\pi = \exp(-D^2/\sigma^2)$, $\sigma = 0.75 \cdot \sqrt{3}$.

Algorithm 1 sparse linear explanations

Step 3 K-LASSO, then the weights

Applicant x: 46 kCHF · 0.66 · 3 yrs → $f(x) = 0.004$ → DENIED

	z' (on / off)			z (actual input to f)			f(z)	D	π
	inc	debt	yrs	inc	debt	yrs			
x'	1	1	1	46.0	0.66	3.0	0.004	0.00	1.00
z' ₁	0	1	1	85.1	0.66	3.0	0.548	1.00	0.55
z' ₂	1	0	1	46.0	0.14	3.0	0.950	1.00	0.55
z' ₃	1	1	0	46.0	0.66	8.8	0.038	1.00	0.55
z' ₄	0	0	1	53.6	0.78	3.0	0.004	1.41	0.31

Require: Classifier f , Number of samples N
Require: Instance x , and its interpretable version x'
Require: Similarity kernel π , Length of explanation K

```
Z ← { }  
for i ∈ {1, 2, 3, ..., N} do  
    z'□ ← sample_around( x' )  
    Z ← Z ∪ { z'□ , f(z□) , π□(z□) }  
end for  
  
w ← K-Lasso( Z , K )  
    ► features: z'□    target: f(z)  
return w
```

Row z'_2 already explains the denial on its own: resample debt to 0.14 and the model jumps **0.004** → **0.950**.

K-LASSO — two stages, two jobs

Lasso chooses. Least squares measures.

1 Choose which K

Run Lasso with a huge penalty λ : every coefficient is zero. Lower λ slowly. Coefficients wake up one at a time, strongest first. **Stop the moment exactly K are non-zero.**

Those K are your features. (This is the "regularization path".)

2 Size them

Throw the Lasso weights away. Refit plain weighted least squares on just those K features.

Those numbers are what you report.

Why two stages? Lasso is good at **choosing** and bad at **measuring** its penalty shrinks every coefficient toward zero, so the magnitudes are biased.

So we use it to choose, then use least squares to measure. In one sentence: **Lasso is a trick for picking K features without trying every combination.**

Where the cost actually is: the regression takes milliseconds. The N calls to f do not. Random forest, N = 5 000: **under 3 seconds**. One Inception image: **~10 minutes**.

What is returned?

w: one number per switch

w is the explanation of this one prediction

Text

w_j = the weight of word j.
Sign = direction, size = strength.
Show the K largest $|w_j|$.

Image

w_j = the weight of super-pixel j.
Highlight the K with the largest
positive weight, grey out the rest.

Tabular

w_j = the effect of **keeping**
feature j at his value.
Label the bar with the value.

What is LIME?

Local Interpretable Model-agnostic Explanations

Local surrogate models are interpretable models used to explain individual predictions of black-box models. LIME (Ribeiro, Singh & Guestrin, 2016) fits such a surrogate trained to approximate the black box's predictions in the neighborhood of one instance.



Local

Explains one prediction, in its neighborhood and not the whole model.



Interpretable

The surrogate is a simple, sparse model a human can read.

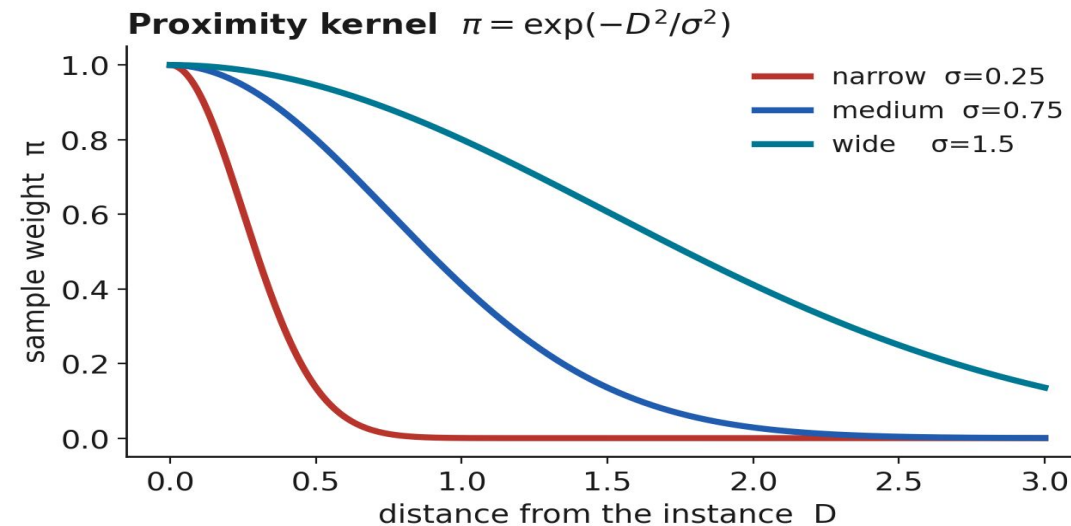


Model-agnostic

Treats the black box as a query-only oracle.

Proximity Kernel

Same objective, three kernel widths → different notions of “local”.



π_x defines “nearby”. The proximity kernel decides which samples count

LIME for text: which words drove it?

Interpretable representation = word present / absent

“The plot was **predictable** and **dull**, but the **soundtrack** was genuinely **beautiful**.”

■ pushes toward NEGATIVE ■ pushes toward POSITIVE

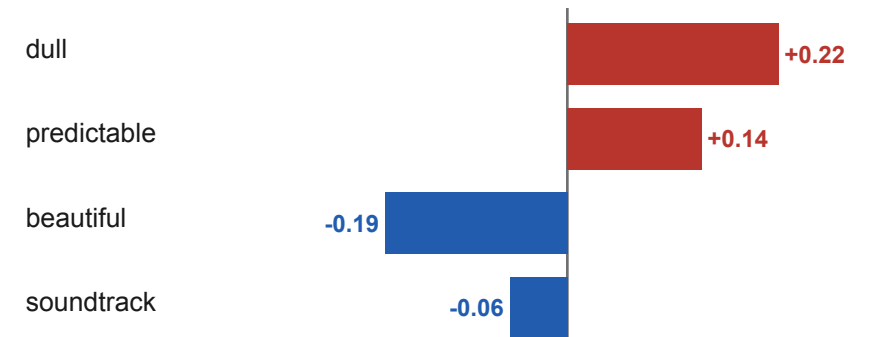
Prediction
NEGATIVE
p = 0.63

Mechanism: toggle words on / off, re-query the model

	dull	predictable	beautiful	soundtrack	p(neg)
x' (original)	1	1	1	1	0.63
z'₁ – dull	0	1	1	1	0.41
z'₂ – beautiful	1	1	0	1	0.79
z'₃ – both	0	1	0	1	0.55

Remove “dull” → less negative. Remove “beautiful” → more negative.

The explanation: word weights



+ toward NEGATIVE · - toward POSITIVE

LIME for images: turn super-pixels on / off

The interpretable representation is a binary super-pixel mask



1 Original

the raw image the black box scored.



2 Super-pixels

group neighbouring pixels into ~40 patches, the on/off units.



3 Perturb

grey out a random subset (1 = shown, 0 = greyed) re-query the model.



4 Explain

patches whose presence most raised the score = the explanation.

LIME in numbers: one explanation, end to end

A real gradient-boosting loan model · one applicant · every step with actual numbers

1 Applicant x: income **46 kCHF** · debt ratio **0.66** · 3 yrs employed

→ black box f →

$f(x) = 0.004$ → DENIED

2 Perturb → query → weight

5 000 samples drawn 5 shown

	z' (on / off)			z (actual input to f)			f(z)	D	π
	inc	debt	yrs	inc	debt	yrs			
x'	1	1	1	46.0	0.66	3.0	0.004	0.00	1.00
z' ₁	0	1	1	85.1	0.66	3.0	0.548	1.00	0.55
z' ₂	1	0	1	46.0	0.14	3.0	0.950	1.00	0.55
z' ₃	1	1	0	46.0	0.66	8.8	0.038	1.00	0.55
z' ₄	0	0	1	53.6	0.78	3.0	0.004	1.41	0.31

1 = keep the applicant's value · 0 = resample it · $\pi = \exp(-D^2/\sigma^2)$, $\sigma = 0.75 \cdot \sqrt{3}$

3 Fit → the explanation

weighted ridge on z' → coefficients

debt_ratio = 0.66

 -0.306

income = 46 kCHF

 -0.174

years_empl. = 3

 -0.068

All three negative — each one is holding this application down.

Each coefficient = the effect of **keeping** that feature at the applicant's value. Debt ratio 0.66 is the single biggest reason for the denial (-0.31)
row z'_2 shows it directly: resample debt to 0.14 and the model jumps 0.004 → 0.950.

The LIME recipe

Five steps — K-LASSO is the concrete default (Algorithm 1)

- 1 Represent** Take the instance in its interpretable form x' — a binary on/off vector (word present, super-pixel shown, feature bucket).
- 2 Perturb** Sample many z' by switching components of x' on/off.
- 3 Query** Map each z' back to raw input z and record the black box's prediction $f(z)$.
- 4 Weight** Weight each sample by proximity π to the original instance.
- 5 Fit (K-LASSO)** Fit a sparse linear model; select K features with LASSO along the regularization path, then least-squares the weights.

Molnar vs. paper: Molnar calls LASSO “a good choice” (among forward/backward selection); the paper names K-LASSO as *the* default procedure. Same idea, no contradiction.

06

Limitations of LIME

- How local is “local”? · instability.
- The cracks that motivate SHAP and robustness.

Kernel width: how local is “local”?

The proximity kernel $\pi = \exp(-D^2/\sigma^2)$

σ too small → the line chases the local tangent

σ too large distant points pull the line → it spans a curved region → local linearity breaks, signs can flip.

No principled choice tabular default $\approx 0.75 \cdot \sqrt{(\text{\#features})}$
A heuristic, not a derivation.

The robustness problem: the same point yields qualitatively different explanations and there's no ground truth to arbitrate.

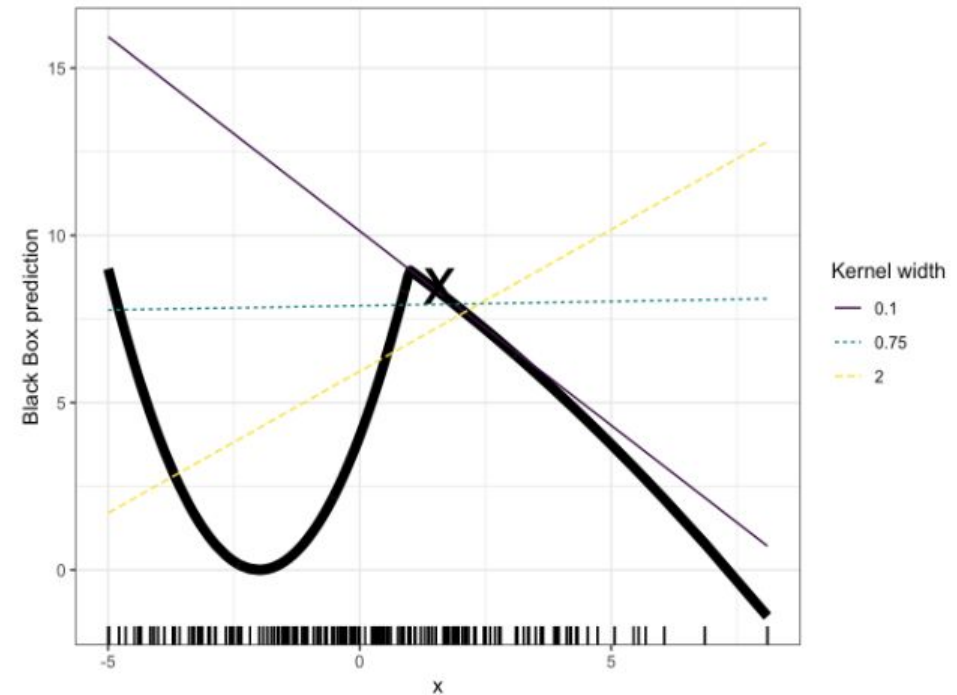


Figure 14.2: Explanation of the prediction of instance $x = 1.6$ with different kernel widths. The model predictions are shown as a thick line and the distribution of the data is shown with rugs. Three local surrogate models with different kernel widths are computed.

Far-away points, instability & other limits

The far-away tension

- By design the kernel makes far points near-weightless
- But widen the neighborhood and those same far points misrepresent the local gradient.
- “Local” and “faithful” pull against each other.

Other practical limits

Instability: random sampling → two runs give different explanations for the same point.

Neighborhood: defining it for tabular data is ad-hoc.

Correlations: perturbations can create unrealistic feature combinations.

Manipulable: explanations can be adversarially fooled

Bridge to SHAP

LIME's open questions

- How wide should the neighborhood be? (kernel width)
- Why these weights, and are they consistent?
- Will I get the same answer twice?

SHAP's answer

- Attributions grounded in game theory (Shapley values).
- Additive, with consistency & local-accuracy guarantees.
- Replaces several of LIME's ad-hoc choices with axioms.

07

From local to global: SP-LIME

- Two trust problems.
- A global picture assembled from local pieces.

Two trust problems

? Trust a prediction

- “Should I act on this individual output?”
 - Solved by one local, faithful, sparse explanation.
- this is LIME.

? Trust the model

- “Will it behave reasonably once deployed?”
 - You can’t read thousands of local explanations.
 - Users have a limited budget B of explanations to inspect.
- this needs SP-LIME.

SP-LIME: submodular pick

A representative handful of local explanations

Goal: pick B instances whose explanations are maximally covering and non-redundant.

How: frame it as a submodular coverage problem: reward covering important features not yet seen.

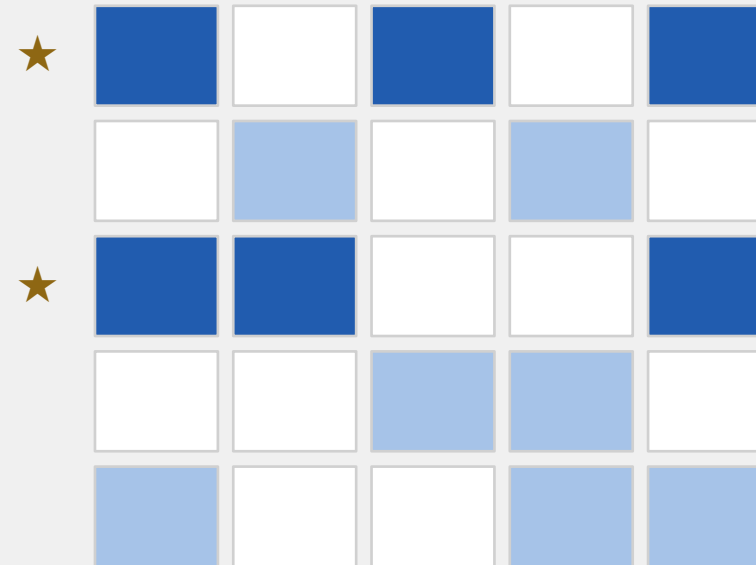
Solve: greedy selection, with the classic $1 - 1/e$ approximation guarantee.

Crucial: each explanation stays strictly *local*.

“Global” means a curated *collection* of local explanations

instances ↓

Pick rows that cover the most (weighted) columns



features →

★ picked instances cover the important features with no overlap.

08

Robustness & certification

- Can we trust the explanation itself?
- From attacks to provable guarantees.

How LIME is used today

From research idea to everyday XAI tool



Model debugging

Surface spurious shortcuts and data leakage before shipping.



NLP

Token-level attributions for text classifiers and, increasingly, LLM outputs.



Vision

Super-pixel explanations for image models what region drove the label.



Regulated domains

Per-decision justification in credit, insurance, healthcare.

Robustness of explanations

Can we trust the explanation itself?

Definition: an explanation is **robust** / **stable** if small, meaning-preserving changes to the input, or just re-running the sampler don't materially change it.

Why LIME is fragile

Sampling randomness: two runs, same point, different coefficients.

Kernel width: the σ choice changes which features look important.

Off-manifold samples: perturbations land where no real data lives.

How the field measures it

Local Lipschitz: bound how much the explanation can move per unit input change (Alvarez-Melis & Jaakkola, 2018).

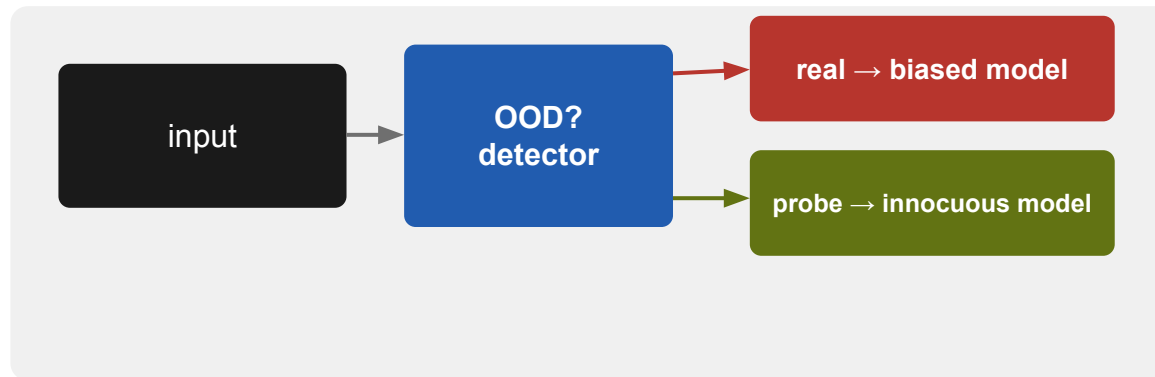
Stability indices: repeat runs, measure variance (OptiLIME, DLIME: re-run the sampler, and see how much the answer moves anyway. Variants like OptiLIME and DLIME then try to fix it by tuning sigma, or by making the sampling deterministic.).

Attack: fooling LIME & SHAP

Slack, Hilgard, Jia, Singh & Lakkaraju (2020)

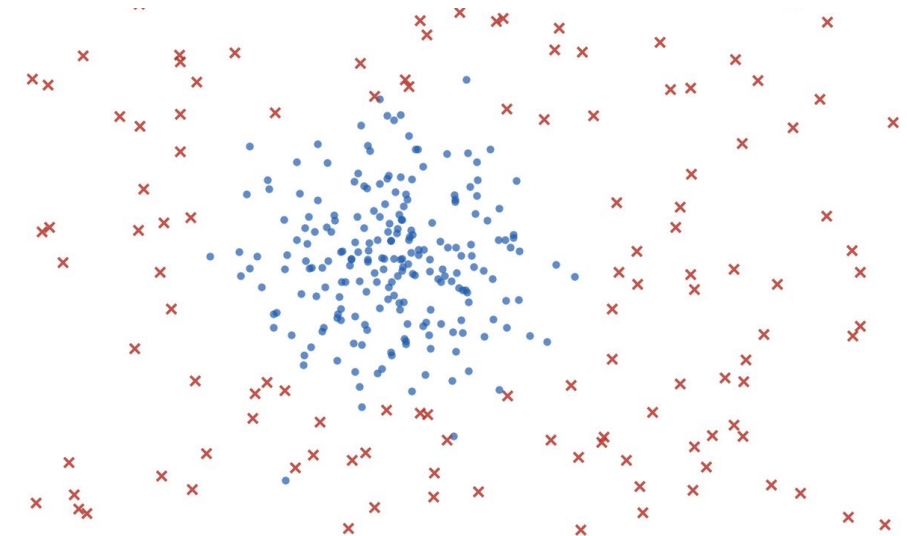
The weakness: LIME probes the model with perturbations that don't look like real data they sit off the data distribution.

The scaffold: an adversary wraps a biased model in a detector: “is this a real input, or a LIME probe?”



The attack perturbations don't look like real data

- real data (on-distribution)
- ✗ LIME perturbations (off-distribution)



Result: LIME only ever sees the innocuous behavior → the explanation looks fair while the deployed model discriminates.

From robustness to certification

Provable guarantees, borrowed from adversarial robustness

Certification: don't just measure stability but **prove** it. Guarantee that the set of important features won't change under any small perturbation within a radius r .

Hard stability

- Certifies that ALL perturbations within radius r keep the prediction.
- A strong, worst-case guarantee.
- Achieved via smoothing (e.g. MuS, NeurIPS 2023) but conservative.

Soft (probabilistic) stability

- Quantifies HOW OFTEN the prediction is maintained under perturbation.
- Model-agnostic, sample-efficient to certify (SCA).
- Finer-grained: tells apart two explanations that look identical.

Jin et al. (2025), "Probabilistic Stability Guarantees for Feature Attributions"; MuS — Xue et al. (2023).

Takeaways

1

Right answer $\neq$ right reason

High accuracy hides Clever Hans shortcuts. We need the “why”, not just the “what”.

2

LIME = local surrogate

Perturb the interpretable representation locally, weight by proximity, fit a sparse model. Faithful locally, silent globally.

3

Local $\rightarrow$ global via SP-LIME

A representative, non-redundant handful of local explanations lets even non-experts pick the better model.

4

Fragility $\rightarrow$ robustness & certification

Kernel width, instability and adversarial attacks

Thank you

Questions & discussion

Key references

Ribeiro, Singh & Guestrin (2016) “Why Should I Trust You?”, KDD. · Molnar, Interpretable Machine Learning.

Slack et al. (2020), Fooling LIME and SHAP, AIES. · Alvarez-Melis & Jaakkola (2018). · Jin et al. (2025); Xue et al. (2023), MuS.